

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO PROJECT

Hệ thống phân tích, dự báo và trực quan hóa thời tiết

HỌC PHẦN: LƯU TRỮ VÀ XỬ LÝ DỮ LIỆU LỚN - IT4931

Giảng viên hướng dẫn: PhD Trần Việt Trung

Nhóm thực hiện: Nhóm 4

Thành viên: Lê Hoài Nam – 20235173

Ngô Đình Phong – 20235186

Đào Duy Anh – 20235000

Nguyễn Tùng Dương – 20230024

Nguyễn Trọng Tài – 20235213

Mã lớp: 168482

Hà Nội, tháng 6 năm 2025

Mục lục

1	ĐỊNH NGHĨA BÀI TOÁN	3
1.1	Bài toán lựa chọn	3
1.2	Phân tích mức độ phù hợp với dữ liệu lớn	3
1.3	Phạm vi và giới hạn dự án	4
2	KIẾN TRÚC VÀ THIẾT KẾ	4
2.1	Kiến trúc tổng thể	4
2.2	Crawler Layer	4
2.3	Speed Layer	5
2.3.1	Luồng Dữ liệu và Các pha xử lý	6
2.3.2	Cơ chế chịu lỗi và Triển khai	6
2.4	Batch Layer	6
2.4.1	Luồng Dữ liệu và Các Kỹ thuật Xử lý	7
2.4.2	Lưu trữ và Xuất Dữ liệu	7
2.5	Serving Layer	8
2.5.1	Vai trò và chức năng cốt lõi	8
2.5.2	Cơ chế truy vấn và trực quan hóa	8
2.6	Airflow	9
2.7	Deploy: Google Cloud Platform	10
3	CHI TIẾT TRIỂN KHAI	10
3.1	Mã nguồn và tài liệu kỹ thuật	10
3.2	File cấu hình	10
4	BÀI HỌC KINH NGHIỆM	11
4.1	Bài học 1: Xử lý giới hạn request	11
4.2	Bài học 2: Tối ưu chi phí và quản lý Partition khi ghi MinIO	12
4.3	Bài học 3: Đồng bộ hóa State giữa Kafka và Checkpointing Spark Streaming	13
4.4	Bài học 4: Tách biệt dữ liệu và định dạng lưu trữ	14
4.5	Bài học 5: Điều phối Container và quản lý biến môi trường	15
4.6	Bài học 6: Tái cấu trúc logic Unpivot dữ liệu với Spark	15
4.7	Bài học 7: Thiết lập cơ chế giám sát tự động	17



4.8	Bài học 8: Quản lý bộ nhớ và tối ưu tính toán phân tán cho Spark Batch .	18
4.9	Bài học 9: Quản lý Secret Keys	19
4.10	Bài học 10: Khôi phục Streaming khi bị dừng đột ngột	20
4.11	Bài học 11: Quản trị định danh	20

1 ĐỊNH NGHĨA BÀI TOÁN

1.1 Bài toán lựa chọn

Hệ thống được thiết kế nhằm mục đích thu thập, tiền xử lý, lưu trữ và phân tích dữ liệu thời tiết thực tế tại các tỉnh thành phố của Việt Nam theo thời gian thực và lịch sử. Dữ liệu thời tiết thể hiện sự hữu ích trong cuộc sống hàng ngày đồng thời đóng vai trò quan trọng trong việc đưa ra các cảnh báo sớm về các hiện tượng cực đoan (nắng nóng gay gắt, mưa lớn gây ngập lụt, hạn hán), hỗ trợ đắc lực cho công tác quản lý tài nguyên, quy hoạch nông nghiệp và phòng chống thiên tai.

Hệ thống giải quyết hai luồng nghiệp vụ chính:

- **Luồng streaming:** Cung cấp các chỉ số thời tiết hiện tại (nhiệt độ, độ ẩm, lượng mưa, tốc độ gió) tức thời cho các tỉnh thành để đưa ra cảnh báo thời tiết khẩn cấp.
- **Luồng batch:** Tính toán các chỉ số thống kê cực trị dài hạn, phân tích so sánh cùng kỳ năm trước, tính toán chuỗi ngày nắng nóng liên tục, và phân tích xu hướng nhiệt độ trung bình tháng của các địa phương qua các năm.

1.2 Phân tích mức độ phù hợp với dữ liệu lớn

Hệ thống đáp ứng toàn diện 5 đặc trưng cơ bản của dữ liệu lớn:

- **Volume:** Hệ thống có thể xử lý lượng lớn dữ liệu bao gồm dữ liệu thời tiết hiện tại, dự báo 15 ngày tới và dữ liệu lịch sử tại các tỉnh thành.
- **Velocity:** Hệ thống có thể đảm bảo luồng dữ liệu tốc độ cao nhờ Apache Kafka và Spark Structured Streaming xử lý và ghi nhận dữ liệu thời gian thực.
- **Variety:** Xử lý hiệu quả dữ liệu nhờ khả năng biến đổi schema linh hoạt của Spark SQL và kho lưu trữ NoSQL Elasticsearch.
- **Veracity:** Dữ liệu được xử lý qua các luồng Spark Batch và Streaming (xử lý giá trị null, lọc trùng lặp, chuẩn hóa định dạng ngày giờ/múi giờ) trước khi lưu trữ.
- **Value:** Cung cấp cảnh báo sớm thời tiết cực đoan thông qua giao diện giám sát thời gian thực, đồng thời làm nền tảng phân tích xu hướng khí hậu dài hạn.

1.3 Phạm vi và giới hạn dự án

Hệ thống tự động thu thập và xử lý dữ liệu thời tiết của các tỉnh/thành phố trên toàn quốc rồi hiển thị thông qua dashboard với các thông tin sau:

- *Thời tiết hiện tại*: Hiển thị nhiệt độ, nhiệt độ cảm nhận, độ ẩm, lượng mưa, tốc độ/hướng gió, áp suất, độ bao phủ mây, tầm nhìn, chỉ số UV, và giờ mặt trời mọc/lặn trong thời điểm hiện tại.
- *Dự báo thời tiết*: Cung cấp dự báo chi tiết theo giờ trong ngày và dự báo dài hạn trong 10 ngày tới bao gồm khoảng nhiệt độ cao nhất/thấp nhất, xác suất mưa và mô tả điều kiện thời tiết.
- *Thống kê thời tiết*: Hiển thị ngày nóng nhất/lạnh nhất, tính toán chuỗi đợt nắng nóng kéo dài (số ngày liên tục, khoảng thời gian xảy ra, đỉnh nhiệt độ) và nhiệt độ trung bình của mỗi tháng qua từng năm.
- *Biểu đồ dữ liệu lịch sử*: Trực quan hóa xu hướng biến đổi của nhiệt độ, lượng mưa, và độ ẩm theo các khung thời gian linh hoạt.

2 KIẾN TRÚC VÀ THIẾT KẾ

2.1 Kiến trúc tổng thể

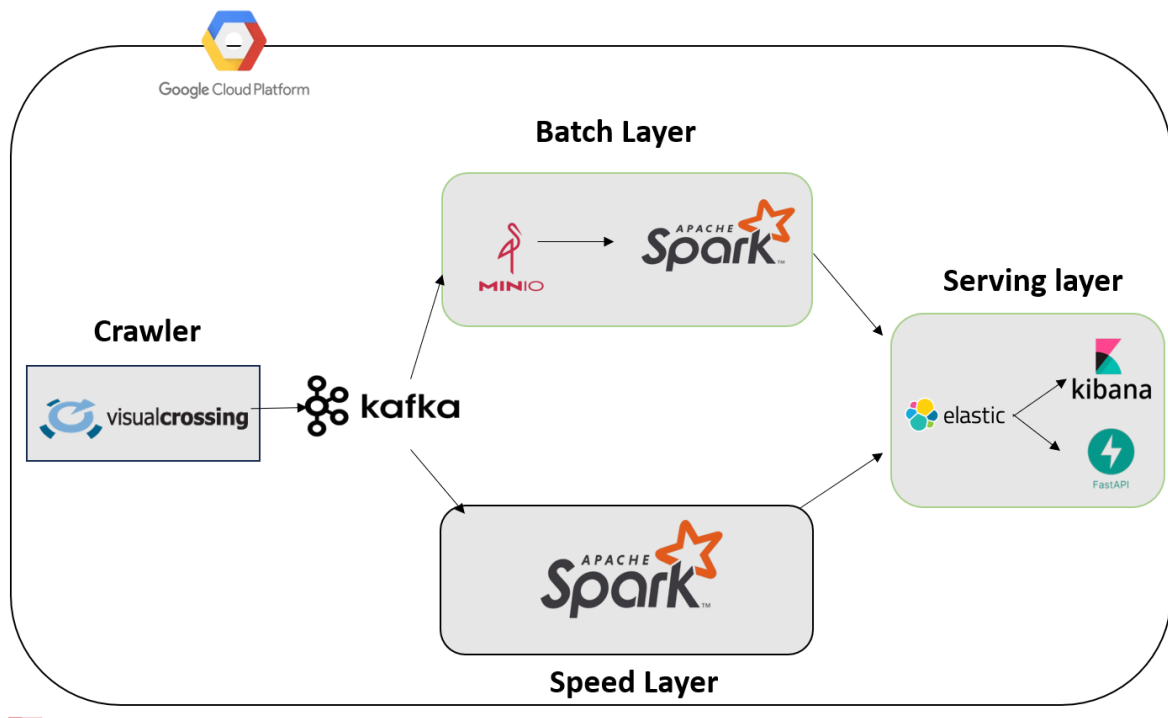
Hệ thống áp dụng kiến trúc **Lambda** để kết hợp hài hòa ưu điểm của cả xử lý Batch và xử lý Streaming.

2.2 Crawler Layer

Thành phần Crawler chịu trách nhiệm thu thập tự động dữ liệu từ các API công cộng rồi đẩy vào Apache Kafka. Hệ thống được điều phối bởi Apache Airflow: Sử dụng *Visual Crossing Weather API* để lấy dữ liệu thời tiết hiện tại và dự báo. Hệ thống tự động xoay vòng danh sách các API Keys (`VISUAL_CROSSING_API_KEY_LIST`) khi gặp lỗi quá hạn ngạch nhằm đảm bảo quá trình thu thập không bị gián đoạn.

Luồng hoạt động chi tiết của thành phần Crawler được mô tả như sau:

- **Bước 1 - Lấy thông tin vị trí**: Sử dụng *Country State City API* để thu thập danh sách các tỉnh thành của Việt Nam (mã quốc gia VN) bao gồm tên tỉnh, vĩ độ và kinh độ, sau đó lưu trữ cố định vào tệp cấu hình `locations.json`.



Hình 1: Sơ đồ kiến trúc Lambda của hệ thống

- **Bước 2 - Thu thập dữ liệu thô:** Dựa trên tọa độ địa lý lấy ra từ `locations.json`, các tiến trình crawler thực hiện gửi yêu cầu HTTP GET tới các endpoint API để lấy về các gói dữ liệu dạng JSON thô.
- **Bước 3 - Đóng gói và phân phối qua Apache Kafka:** Dữ liệu sau khi thu thập được đóng gói theo định dạng JSON chuẩn hóa và đẩy vào 3 topic riêng biệt trên Apache Kafka phục vụ cho các luồng xử lý tiếp theo:
 - `weather_data`: Lưu trữ thông tin thời tiết thời gian thực (Real-time).
 - `weather_forecast_data`: Lưu trữ thông tin dự báo thời tiết (Forecast).

2.3 Speed Layer

Speed Layer là thành phần xử lý dữ liệu thời gian thực trong kiến trúc Lambda của hệ thống. Thành phần này được xây dựng dựa trên Spark Structured Streaming, có nhiệm vụ đọc luồng dữ liệu từ Kafka, xử lý gần thời gian thực và ghi kết quả vào Elasticsearch. Nhờ đó, Serving Layer có thể truy vấn dữ liệu với tốc độ cao để hiển thị lên Dashboard.

2.3.1 Luồng Dữ liệu và Các pha xử lý

Speed Layer tiếp nhận dữ liệu thô (định dạng JSON) từ hai Kafka topic chính: dữ liệu thời tiết hiện tại (`weather_data`) và dữ liệu dự báo (`weather_forecast_data`). Sau khi dữ liệu được Spark phân giải (parse) theo schema chuẩn, hệ thống định tuyến thành 3 luồng xử lý song song:

- **Luồng Realtime:** Lưu trữ các bản ghi thời tiết hiện tại vào Elasticsearch index `weather_realtime`. Để đảm bảo tính toàn vẹn và tránh ghi trùng lặp dữ liệu khi hệ thống khởi động lại, luồng này sử dụng khóa chính là chuỗi kết hợp giữa tên địa điểm và thời gian cục bộ.
- **Luồng Aggregation:** Nhóm dữ liệu theo từng địa điểm trong các cửa sổ thời gian 6 giờ để tính toán các chỉ số thống kê như: nhiệt độ trung bình/lớn nhất/nhỏ nhất, lượng mưa tổng cộng, và sức gió tối đa. Luồng này áp dụng *Watermark* với độ trễ 2 giờ để xử lý các dữ liệu đến muộn. Kết quả được cập nhật vào index `weather_aggregated_6h`.
- **Luồng Forecast:** Trích xuất và chuẩn hóa thông tin dự báo thời tiết 10 ngày tới, sau đó ghi trực tiếp vào index `weather_forecast`.

2.3.2 Cơ chế chịu lỗi và Triển khai

Để đảm bảo tính bền bỉ và khả năng phục hồi, Speed Layer áp dụng cơ chế Checkpoint của Spark. Dữ liệu checkpoint (chứa trạng thái xử lý và offset của Kafka) được mount ra ngoài thông qua Docker Volume. Nhờ thiết kế này, nếu container Spark bị gián đoạn hoặc khởi động lại, ứng dụng có thể đọc lại offset và tiếp tục xử lý chính xác từ vị trí dừng trước đó mà không làm mất mát hay sai lệch dữ liệu.

Trong kiến trúc triển khai thực tế bằng Docker Compose, Speed Layer được cấu hình để hoạt động đồng bộ, tự động chờ trạng thái sẵn sàng (healthy) từ cụm Kafka và Elasticsearch trước khi bắt đầu luồng streaming.

2.4 Batch Layer

Batch Layer đảm nhiệm vai trò xử lý khối lượng lớn dữ liệu lịch sử trong kiến trúc Lambda. Thành phần này được xây dựng trên nền tảng Apache Spark, có nhiệm vụ trích xuất dữ liệu thô từ kho lưu trữ MinIO, tiến hành làm sạch, chuẩn hóa và thực hiện các phép

tổng hợp phức tạp trước khi tải kết quả vào Elasticsearch để Serving Layer truy vấn và hiển thị lên Dashboard.

2.4.1 Luồng Dữ liệu và Các Kỹ thuật Xử lý

Batch Layer đọc dữ liệu lịch sử từ MinIO và thực hiện chuỗi các bước biến đổi dữ liệu chuyên sâu, bao gồm:

- **Làm sạch và Chuẩn hóa:** Trích xuất và định dạng lại các trường thông tin quan trọng như địa điểm, thời gian, nhiệt độ, độ ẩm và lượng mưa, đồng thời tự động loại bỏ các bản ghi không hợp lệ.
- **Xử lý Sáp nhập Hành chính:** Giải quyết bài toán thay đổi đơn vị hành chính từ 63 tỉnh/thành cũ xuống còn 34 tỉnh/thành mới. Hệ thống ứng dụng kỹ thuật Broadcast Join để phân phối file cấu hình mapping tới các node xử lý, giúp tránh hiện tượng thất cổ chai khi xử lý dữ liệu lớn. Kết quả tạo ra hai góc nhìn dữ liệu song song: trước sáp nhập (`pre_merge_63`) và sau sáp nhập (`post_merge_34`).
- **Tổng hợp dữ liệu theo Ngày:** Nhóm dữ liệu theo ngày và địa điểm để tính toán các chỉ số cơ bản (nhiệt độ trung bình/cao nhất/thấp nhất, tổng lượng mưa, độ ẩm trung bình). Dữ liệu này làm cơ sở để phân loại mức độ cảnh báo thời tiết từ bình thường đến cực đoan.
- **Phân tích Cùng kỳ:** Sử dụng Sort-Merge Join để kết nối dữ liệu của năm hiện hành với dữ liệu cùng kỳ năm trước, cho phép tính toán chính xác mức độ chênh lệch của các chỉ số thời tiết.
- **Phát hiện Đợt Nắng Nóng:** Ứng dụng các hàm cửa sổ để theo dõi, đánh dấu và cộng dồn các ngày có nhiệt độ vượt ngưỡng, từ đó xác định độ dài của chuỗi ngày nắng nóng liên tiếp dài nhất tại từng địa phương.
- **Chuyển đổi Cấu trúc (Pivot/Unpivot):** Xoay trục dữ liệu nhiệt độ trung bình theo 12 tháng trong năm (Pivot) để tạo cái nhìn tổng quan, sau đó giải nén (Unpivot) ngược lại thành cấu trúc dạng dòng, đi kèm thứ tự tháng để tối ưu cho việc truy vấn và sắp xếp trên Elasticsearch.

2.4.2 Lưu trữ và Xuất Dữ liệu

Để tối ưu hóa kiến trúc, dữ liệu đầu ra của Batch Layer được xuất đồng thời ra hai đích đến:

- **Cơ sở dữ liệu Elasticsearch:** Các bảng dữ liệu tổng hợp (thống kê ngày, YoY, cực trị thời tiết, dữ liệu unpivot) được ghi vào các index tương ứng thông qua Elasticsearch Spark Connector.
- **Kho lưu trữ MinIO:** Dữ liệu đã qua xử lý được lưu ngược lại Data Lake MinIO dưới định dạng Parquet và được phân vùng (partition) theo năm. Cách tiếp cận này giúp tối ưu hóa chi phí lưu trữ và tăng tốc độ đọc dữ liệu nếu cần chạy lại các luồng phân tích trong tương lai.

2.5 Serving Layer

Serving Layer là tầng cuối cùng trong kiến trúc Lambda, đóng vai trò như một cầu nối giao tiếp giữa kho dữ liệu đã qua xử lý Elasticsearch và giao diện người dùng. Thành phần này được phát triển dựa trên FastAPI kết hợp cùng HTML/CSS/JavaScript, nhằm cung cấp các API truy xuất dữ liệu từ thời gian thực đến lịch sử dài hạn.

2.5.1 Vai trò và chức năng cốt lõi

Khác với Batch và Speed Layer, Serving Layer không thực hiện các tác vụ tính toán hay xử lý dữ liệu nặng. Nhiệm vụ chính của tầng này được tinh gọn bao gồm:

- Tiếp nhận các request truy vấn từ giao diện người dùng.
- Đóng gói và xây dựng các truy vấn động (*Elasticsearch Query DSL*) dựa trên các tham số bộ lọc (địa điểm, thời gian, góc nhìn).
- Truy xuất dữ liệu tốc độ cao từ các index đã được chuẩn bị sẵn trong Elasticsearch, bao gồm dữ liệu realtime, dự báo, thống kê cực trị, so sánh YoY và dữ liệu phân tích tháng.
- Chuẩn hóa cấu trúc dữ liệu trả về dưới định dạng JSON phục vụ cho các thư viện render biểu đồ.

2.5.2 Cơ chế truy vấn và trực quan hóa

Để đáp ứng các kịch bản phân tích phức tạp trên Dashboard, Serving Layer cung cấp các luồng API linh hoạt:

- **Truy vấn thời gian thực & dự báo:** Lấy chính xác bản ghi thời tiết mới nhất và dự báo nhiều ngày tiếp theo thông qua việc đối chiếu ID địa phương.

- **Bộ lọc theo góc nhìn Hành chính:** Hỗ trợ truy vấn dữ liệu Batch một cách liên mạch giữa hai chế độ hiển thị: 63 tỉnh/thành trước sáp nhập (`pre_merge_63`) và 34 tỉnh/thành sau sáp nhập (`post_merge_34`).
- **Phục vụ phân tích Pivot/Unpivot:** Truy vấn trực tiếp vào index `weather_batch_unpivoted` để lấy dữ liệu nhiệt độ trung bình theo từng tháng trong năm. Dữ liệu này sau đó được đẩy xuống client, kết hợp cùng các thư viện đồ họa như `Chart.js` để tự động vẽ biểu đồ cột và render các bảng Pivot tổng hợp.

2.6 Airflow

Trong dự án này, Apache Airflow đóng vai trò là bộ điều phối. Thành phần này chịu trách nhiệm tự động lập lịch và kích hoạt các tác vụ thu thập dữ liệu bằng cách khởi tạo các container Docker độc lập thông qua `DockerOperator`.

Hệ thống được thiết kế với 2 luồng công việc (DAGs) chính:

- `daily_forecast_pipeline` (Chạy định kỳ lúc 00:00 hàng ngày): Xóa trạng thái cũ và kích hoạt tiến trình cào dữ liệu dự báo thời tiết (`weather_crawler.py --type fcst`).
- `realtime_environment_pipeline` (Chạy mỗi 30 phút, cụ thể ở phút thứ 5 và 35 của mỗi giờ): Kích hoạt tiến trình cào dữ liệu thời tiết hiện tại (`weather_crawler.py --type current`) và dữ liệu ô nhiễm không khí (`pollution_crawler.py`).

Bên cạnh chức năng lập lịch, cơ chế quản lý trạng thái là một giải pháp thiết kế trọng yếu nhằm giải quyết các rào cản của hệ thống phân tán.

Do các tác vụ thu thập dữ liệu được thực thi bên trong các container có vòng đời ngắn và tự động tiêu hủy ngay sau khi hoàn thành (thông qua tham số `auto_remove='force'`), hệ thống đòi hỏi một cơ chế lưu trữ bền vững để duy trì ngữ cảnh hoạt động xuyên suốt. Giải pháp được triển khai là tích hợp một phân vùng lưu trữ ảo dùng chung (Named Volume) mang định danh `crawler_state_vol`.

Cơ chế ánh xạ dữ liệu này vận hành theo hai nguyên tắc đảm bảo tính toàn vẹn:

- **Bảo toàn trạng thái:** Tập dữ liệu `weather_crawler_state.json` được lưu trữ tập trung trên Volume chung và ánh xạ trực tiếp vào không gian làm việc `/app/state` của mọi container thu thập dữ liệu mỗi khi chúng được cấp phát khởi chạy.
- **Kiểm soát phân quyền:** Để ngăn chặn sự cố xung đột quyền sở hữu người dùng

giữa các tệp sinh ra bởi container (sở hữu bởi đặc quyền Root) và tiến trình lỗi của Airflow, hệ thống triển khai một tác vụ độc lập mang tên `clean_old_state_task`. Tác vụ này được cô lập trên môi trường Alpine Linux, vận hành với đặc quyền Root để chủ động dọn dẹp và làm mới trạng thái hệ thống vào đầu mỗi chu kỳ ngày một cách an toàn.

2.7 Deploy: Google Cloud Platform

- Hệ điều hành: Ubuntu 26.04 LTS.
- RAM: 16GB.
- Số lượng vCPU: 4.
- Link: [Weather Forecast](#)

3 CHI TIẾT TRIỂN KHAI

3.1 Mã nguồn và tài liệu kỹ thuật

Mã nguồn hệ thống: [Namletapcode/Big-Data](#)

Di chuyển vào thư mục `docker_deployment` và chạy tập lệnh khởi tạo tương ứng với hệ điều hành của bạn:

Đối với Windows:

```
1 cd docker_deployment
2 deploy.bat
```

Đối với macOS / Linux: (Lưu ý: Cần cấp quyền thực thi cho file script ở lần chạy đầu tiên bằng lệnh `chmod +x`)

```
1 cd docker_deployment
2 chmod +x deploy.sh
3 ./deploy.sh
```

3.2 File cấu hình

File cấu hình: [Link Google Drive](#)

Tải về và đưa file `crawler.env` vào thư mục `crawler`, các file còn lại vào thư mục `docker_deployment`.

4 BÀI HỌC KINH NGHIỆM

4.1 Bài học 1: Xử lý giới hạn request

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Hệ thống cần liên tục cào dữ liệu từ Visual Crossing để lấy thông tin thời tiết cho các tỉnh thành phố.
- **Thách thức gặp phải:** Các API công cộng giới hạn số lượng request.
- **Tác động đến hệ thống:** Nếu bị block, hệ thống bị khuyết dữ liệu, gây nhiễu cho luồng xử lý và Dashboard hiển thị sai lệch thông tin thực tế.

Cách tiếp cận đã thử

- Cố định một tài khoản API trả phí cao để dùng chung. *Đánh đổi:* Chi phí vượt ngân sách và vẫn có khả năng bị nghẽn nếu tăng tần suất lấy dữ liệu lên thời gian thực.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Xây dựng cơ chế "Xoay vòng API Keys tự động" trong mã nguồn (`weather_fetcher.py`). Hệ thống lưu trữ danh sách key dạng mảng, sử dụng biến đếm index lưu vào tệp trạng thái (`weather_crawler_state.json`) và tự động chuyển sang key tiếp theo khi gặp HTTP 429/401.
- **Kết quả:** không bị chặn khi lấy dữ liệu do rate limit

Điểm rút ra

Luôn dự phòng cơ chế xoay vòng hoặc exponential backoff khi dùng API của bên thứ ba.

4.2 Bài học 2: Tối ưu chi phí và quản lý Partition khi ghi MinIO

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Lớp Batch sử dụng Spark để đọc dữ liệu lịch sử thô dạng JSON từ MinIO, tổng hợp các chỉ số (nhiệt độ, cảnh báo, YoY) và lưu lại lên MinIO dạng Parquet.
- **Thách thức gặp phải:** Dữ liệu JSON thô có dung lượng nhỏ nhưng rất nhiều tệp. Việc đọc mất nhiều thời gian, ghi xuất file bị xé nhỏ, dẫn đến tiêu thụ nhiều RAM và chậm I/O trên cụm Spark container.
- **Tác động đến hệ thống:** Job Batch bị chậm đáng kể, rủi ro Out Of Memory, tăng chi phí tính toán.

Cách tiếp cận đã thử

- **Cách 1:** Sử dụng `df.repartition(1)` trước khi ghi ra để ép thành 1 file. *Đánh đổi:* Mất đi tính toán song song (parallelism), job chạy dồn vào 1 worker gây thất cổ chai cục bộ và OOM.
- **Cách 2:** Đọc thẳng toàn bộ bucket (wildcard) không phân trang. *Đánh đổi:* Spark mất quá nhiều thời gian list files.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Quản lý Partition theo year và Bucketing theo Location khi ghi ra Parquet để đẩy nhanh tốc độ truy vấn (Partition Pruning).
 - Điều chỉnh `spark.sql.shuffle.partitions = 200` để tận dụng parallelism.
 - Cơ chế Fallback: Nếu không ghi được bằng dạng bucket, tiến hành ghi Parquet phân vùng thông thường thay vì crash.
- **Kết quả:** Thời gian đọc/ghi MinIO bằng Spark giảm mạnh. Các tệp Parquet sinh ra có dung lượng tiêu chuẩn, tối ưu hóa I/O.

Điểm rút ra

Tránh xa "Small Files Problem" khi lưu trữ Big Data. Thay vì ghi dữ liệu theo từng giờ thì hãy sử dụng PartitionBy (năm/tháng) và BucketBy kết hợp định dạng cột (Parquet)

để tối ưu tốc độ truy xuất trong tương lai.

4.3 Bài học 3: Đồng bộ hóa State giữa Kafka và Checkpointing Spark Streaming

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Speed Layer dùng Spark Streaming đọc từ Kafka và tổng hợp các cửa sổ 6h trước khi đẩy vào Elasticsearch.
- **Thách thức gặp phải:** Đảm bảo Exactly-once, khi container Spark Streaming restart, nếu không có state rõ ràng, nó có thể đọc lại dữ liệu Kafka gây trùng lặp.
- **Tác động đến hệ thống:** Dữ liệu thời gian thực bị sai lệch thông tin cảnh báo thời tiết.

Cách tiếp cận đã thử

- Cấu hình Kafka `auto.offset.reset = latest`. *Đánh đổi:* Mất dữ liệu khi Spark container sập hoặc restart.

Giải pháp cuối cùng

- **Giải pháp chi tiết & triển khai:**
 - Sử dụng cơ chế Spark Checkpointing độc lập cho từng luồng dữ liệu. Offset của Kafka được Spark tự động lưu vào file checkpoint nội bộ và ánh xạ chính xác với trạng thái xử lý dữ liệu.
 - Xử lý Upsert/Idempotent trên Elasticsearch bằng cách sử dụng Tên Tỉnh + Thời gian làm document ID (`es.mapping.id = es_id`).
- **Kết quả:** Đảm bảo tính nhất quán dữ liệu, Elasticsearch không bao giờ bị ghi trùng lặp bản ghi dù Spark container restart nhiều lần.

Điểm rút ra

Không phụ thuộc vào Kafka offsets để quản lý trạng thái khi dùng Spark Streaming. Kết hợp Checkpointing thư mục và sink idempotent để đạt được Exactly-once.

4.4 Bài học 4: Tách biệt dữ liệu và định dạng lưu trữ

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Hệ thống tạo ra 2 loại dữ liệu: Thời gian thực (phục vụ xem ngay) và lịch sử (phục vụ phân tích/báo cáo).
- **Thách thức gặp phải:** Lưu toàn bộ vào Elasticsearch rất tốn chi phí RAM và Ổ cứng do ES lưu Inverted Index.
- **Tác động đến hệ thống:** Elasticsearch chứa quá nhiều dữ liệu lịch sử sẽ tràn RAM, tràn ổ và sập.

Cách tiếp cận đã thử

- Lưu 100% dữ liệu vào Elasticsearch. *Đánh đổi:* Yêu cầu mở rộng cấu hình phần cứng lớn, lãng phí chi phí RAM cho các dữ liệu ít truy cập.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:** Áp dụng kiến trúc *Lambda Storage*.
 - Sử dụng Elasticsearch là "Hot Storage" lưu dữ liệu real-time, tổng hợp nhanh, phục vụ truy vấn tìm kiếm siêu tốc trực tiếp cho API.
 - Sử dụng MinIO S3 Object Storage là "Cold Storage" lưu dữ liệu lịch sử kết nối từ Kafka qua Kafka Connect. Dữ liệu thô lưu ở dạng JSON, dữ liệu qua xử lý lưu định dạng nén Parquet.
- **Kết quả:** Giảm tải cho Elasticsearch, cho phép mở rộng kho dữ liệu lạnh trên MinIO không giới hạn với chi phí thấp.

Điểm rút ra

Việc chia lớp Hot (ES) và Cold (MinIO/Parquet) giúp hệ thống cân bằng giữa hiệu suất truy vấn và tối ưu hóa chi phí lưu trữ hạ tầng.

4.5 Bài học 5: Điều phối Container và quản lý biến môi trường

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Hệ thống bao gồm nhiều thành phần: Airflow, Spark, Kafka, Zookeeper, Elasticsearch, Kibana, MinIO, FastAPI.
- **Thách thức gặp phải:** Khi gắn cứng cổng giao tiếp và các biến môi trường vào mã nguồn. Khi dịch chuyển từ môi trường Dev sang Production, cổng và biến môi trường có thể thay đổi.
- **Tác động đến hệ thống:** Tốn thời gian để gỡ và build lại image.

Cách tiếp cận đã thử

- Tìm và thay thế chuỗi cổng và biến cục bộ vào file code trước khi build image. *Đánh đổi:* Image không thể tái sử dụng, rủi ro bảo mật cao, tốn công sức manual.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Gom toàn bộ cấu hình vào các file `*.env`. Mã nguồn Python chỉ sử dụng `os.getenv()` để đọc cấu hình.
- **Kết quả:** Hệ thống đạt mức "Build once, run anywhere". Di chuyển từ máy cá nhân lên máy chủ GCP chỉ bằng cách chỉnh sửa file `.env` duy nhất, không cần build lại Image.

Điểm rút ra

Áp dụng nguyên tắc *12-Factor App*: Lưu cấu hình vào biến môi trường. Tận dụng Service Discovery nội bộ của Docker/K8s thay vì chỉ định IP tĩnh.

4.6 Bài học 6: Tái cấu trúc logic Unpivot dữ liệu với Spark

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Serving API cần truy vấn dữ liệu nhiệt độ trung bình 12 tháng của từng tỉnh để vẽ biểu đồ line chart trên UI.

- **Thách thức gặp phải:** Dữ liệu ban đầu được Spark tạo Pivot thành dạng cột ngang (Month_1 đến Month_12). Khi đưa vào UI, frontend gặp khó khăn để ánh xạ danh sách trực.
- **Tác động đến hệ thống:** Nếu Frontend phải vòng lặp map tay, hiệu năng trình duyệt giảm. Nếu Backend vòng lặp parse dữ liệu thì API response bị chậm.

Cách tiếp cận đã thử

- **Cách 1:** Giữ nguyên cấu trúc bảng ngang trên cơ sở dữ liệu, sau đó để máy chủ backend tự chạy vòng lặp chuyển đổi dữ liệu về dạng dòng ngay tại thời điểm người dùng gọi API. *Đánh đổi:* Việc bắt máy chủ "gánh" logic xử lý dữ liệu sẽ làm tiêu tốn rất nhiều tài nguyên CPU, dễ gây nghẽn cổ chai và quá tải hệ thống khi lượng người truy cập tăng vọt.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Đẩy phần xử lý dữ liệu nặng xuống cho tầng Spark Batch thực hiện từ trước. Tại đây, hệ thống sẽ tự động dùng hàm có sẵn của Spark để "xoay" 12 cột dữ liệu nằm ngang (tương ứng với nhiệt độ của 12 tháng) thành 12 dòng dọc. Nhờ cấu trúc được định dạng sẵn này, tầng API chỉ việc lấy ra sử dụng luôn mà không tốn công tính toán lại.
 - Kết quả Unpivot được ghi ra một Index riêng trên ES (`weather_batch_unpivoted`).
- **Kết quả:** FastAPI chỉ việc truy vấn lấy thẳng cấu trúc JSON list phẳng đầy trực tiếp cho Frontend vẽ Echarts, tiết kiệm thời gian xử lý phản hồi API.

Điểm rút ra

Logic xử lý dữ liệu cấu trúc lớn nên được đẩy càng sát về nguồn càng tốt thay vì dồn gánh nặng cho Client.

4.7 Bài học 7: Thiết lập cơ chế giám sát tự động

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Hệ thống phụ thuộc vào sự ổn định của Zookeeper, Kafka và Elasticsearch để duy trì luồng dữ liệu thời gian thực.
- **Thách thức gặp phải:** Đôi khi dịch vụ Zookeeper hoặc Elasticsearch bị quá tải bộ nhớ và treo, nhưng bản thân container không bị thoát. Do đó, Docker không tự động khởi động lại, khiến Spark Streaming không thể ghi hay đọc dữ liệu.
- **Tác động đến hệ thống:** Toàn bộ pipeline bị ngưng trệ nhưng không có cảnh báo. Dữ liệu thời tiết bị gián đoạn và phải tìm lỗi thủ công.

Cách tiếp cận đã thử

- **Cách 1:** Viết script ping cổng mạng (ping port 9092) từ bên ngoài. *Đánh đổi:* Cổng mạng vẫn mở nhưng tiến trình bên trong (ví dụ Kafka Broker) có thể đang bị deadlock, nên việc ping port không phản ánh đúng trạng thái thực tế.

Giải pháp cuối cùng

- **Giải pháp chi tiết & triển khai:**
 - Khai báo thuộc tính `healthcheck` trực tiếp trong `docker-compose.yml` cho các dịch vụ lỗi.
 - Với Zookeeper: Sử dụng lệnh nội trú `echo ruok | nc localhost 2181` (Nếu trả về `imok` là bình thường).
 - Với Kafka: Chạy lệnh `kafka-topics -list` để kiểm tra khả năng phản hồi của Broker.
 - Cấu hình các dịch vụ phụ thuộc (Spark, Kafka Connect) sử dụng `depends_on` với điều kiện `condition: service_healthy`.
- **Kết quả:** Hệ thống tự nhận diện các trạng thái treo giả và tự động tái khởi động container có vấn đề. Các thành phần phụ thuộc chỉ bắt đầu chạy khi dịch vụ lỗi đã thực sự sẵn sàng, loại bỏ hoàn toàn hiện tượng sập ngầm.

Điểm rút ra

Cấu hình `depends_on: service_healthy` là thực hành bắt buộc trong triển khai container hệ phân tán.

4.8 Bài học 8: Quản lý bộ nhớ và tối ưu tính toán phân tán cho Spark Batch

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Batch Layer định kỳ đọc toàn bộ dữ liệu lịch sử từ MinIO để tính toán các bảng tổng hợp (nhiệt độ kỷ lục, chuỗi ngày nắng nóng, YoY).
- **Thách thức gặp phải:** Khi khối lượng dữ liệu tích lũy tăng nhanh qua thời gian, Spark Batch gặp phải hiện tượng nghẽn bộ nhớ, dẫn đến lỗi Out Of Memory Error hoặc thời gian xử lý chậm do quá trình dọn dẹp rác diễn ra liên tục.
- **Tác động đến hệ thống:** Job chạy thất bại, tốn tài nguyên hệ thống, dữ liệu phân tích lịch sử không được cập nhật kịp thời lên Elasticsearch.

Cách tiếp cận đã thử

- **Cách 1:** Cấp phát thêm RAM trực tiếp trong `docker-compose.yml`. *Đánh đổi:* Rất tốn kém về mặt hạ tầng nhưng không giải quyết triệt để khi dữ liệu tiếp tục phình to. Spark vẫn crash nếu một Node phải chịu tải toàn bộ dữ liệu.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Cấu hình lưu trữ trung gian linh hoạt đối với các DataFrame được tái sử dụng nhiều lần (như `daily_df`, `valid_df`) nhằm đảm bảo Spark có thể linh hoạt tràn một phần dữ liệu xuống ổ đĩa thay vì ép buộc nằm hết trên RAM.
 - Cấu hình phân tán tường minh thông số `spark.sql.shuffle.partitions = 200` trong mã nguồn Python, giúp dữ liệu được băm nhỏ đều đặn ra nhiều khối trong các thao tác, tránh quá tải cục bộ.
- **Kết quả:** Năng lực xử lý của hệ thống không còn bị giới hạn cứng bởi RAM. Hệ thống Spark Batch có thể xử lý tập dữ liệu lịch sử hàng năm với cấu hình Container

duy trì ở mức tối thiểu, đảm bảo tính ổn định và tiết kiệm chi phí tính toán đám mây.

Điểm rút ra

Để hệ thống có thể mở rộng một cách bền vững, việc quản lý vòng đời bộ nhớ và băm nhỏ dữ liệu phân tán quan trọng hơn việc chỉ nâng cấp phần cứng.

4.9 Bài học 9: Quản lý Secret Keys

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Kafka, Elasticsearch và MinIO là các công nghệ mở, dễ bị rò rỉ dữ liệu.
- **Thách thức gặp phải:** Nếu commit mật khẩu, API key vào Git Repository, hệ thống có rủi ro bị hacker sử dụng "chùa" API hoặc phá hoại CSDL.
- **Tác động đến hệ thống:** Rủi ro an ninh mạng, bảo mật nghiêm trọng.

Cách tiếp cận đã thử

- **Cách 1:** Lưu trong file config JSON đi kèm code. *Đánh đổi:* Vẫn lộ trên Github/Gitlab khi đẩy code.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Khai báo file `.gitignore` để chặn tất cả file `.env`.
- **Kết quả:** Không có thông tin nhạy cảm bị rò rỉ lên Source Control.

Điểm rút ra

Để hệ thống an toàn, cần tách biệt hoàn toàn mạng nội bộ khỏi Internet. Chỉ cho phép bên ngoài truy cập thông qua một cổng giao tiếp duy nhất và phải bảo vệ nghiêm ngặt, không để lộ các file chứa mật khẩu hoặc cấu hình như `.env`.

4.10 Bài học 10: Khôi phục Streaming khi bị dừng đột ngột

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Các tình huống trực trặc mạng hoặc Spark Streaming restart do Out-of-Memory.
- **Thách thức gặp phải:** Quá trình Ingestion bị đình trệ. Cần cơ chế Resume tiếp tục công việc đang dở mà không bỏ sót hoặc nhân đôi dữ liệu khi hệ thống phục hồi.
- **Tác động đến hệ thống:** Nguy cơ hỏng hóc hoặc mất mát dữ liệu.

Cách tiếp cận đã thử

- **Cách 1:** Xóa Topic và chạy lại Crawler. *Đánh đổi:* Mất hết dữ liệu trong thời gian Downtime, tốn phí gọi lại API.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Sử dụng Spark Streaming Checkpointing và ghi Offsets Topic.
 - Thiết lập Docker Compose Restart Policy: `restart: always` cho các process quan trọng.
- **Kết quả:** Khi hệ thống khôi phục, Spark Streamin tự tìm đến file checkpoint/offset cuối cùng để kéo tiếp lượng dữ liệu tồn đọng mà không bị rớt bản ghi nào.

Điểm rút ra

Message Queue như Kafka đóng vai trò bộ đệm chống sốc, kết hợp quản lý Offset là chìa khóa cho khả năng chịu lỗi và tự phục hồi.

4.11 Bài học 11: Quản trị định danh

Mô tả vấn đề

- **Bối cảnh và nền tảng:** Hệ thống sử dụng cơ chế ổ cứng ảo dùng chung để chia sẻ tệp trạng thái API (`weather_crawler_state.json`) giữa container thu thập dữ liệu và bộ điều phối Airflow.

- **Thách thức gặp phải:** Xung đột quyền sở hữu người dùng theo chuẩn POSIX của Linux. Container Python sinh ra tệp dữ liệu dưới đặc quyền cao nhất (Root - UID 0), trong khi tiến trình dọn dẹp của Airflow lại bị giới hạn ở tài khoản người dùng an toàn (UID 1001 hoặc 50000). Điều này dẫn đến lỗi `Permission denied` khi Airflow cố gắng xóa tệp trạng thái cũ.
- **Tác động đến hệ thống:** Quá trình dọn dẹp trạng thái bị từ chối, khiến các chu kỳ thu thập dữ liệu của ngày hôm sau không thể tái lập lượng quota API, dẫn đến nguy cơ đình trệ toàn bộ pipeline.

Cách tiếp cận đã thử

- **Cách 1:** Sử dụng `BashOperator` gọi lệnh xóa trực tiếp kèm cờ bỏ qua lỗi (`rm -f || true`). *Đánh đổi:* Lỗi bị che giấu, tệp cũ do Root sở hữu vẫn tồn tại nguyên vẹn, logic luân phiên API Key thất bại.
- **Cách 2:** Cấp quyền `rwx` tối đa (`chmod 777`) cho tệp từ bên trong mã nguồn Python. *Đánh đổi:* Vi phạm nguyên tắc đặc quyền tối thiểu trong bảo mật hệ thống.

Giải pháp cuối cùng

- **Giải pháp chi tiết & Triển khai:**
 - Áp dụng mẫu thiết kế "Cô lập đặc quyền" bằng cách sử dụng `DockerOperator` thay vì ép Airflow tự xử lý.
 - Khởi tạo một container (`alpine:latest`) vận hành dưới quyền Root, được ánh xạ vào đúng Volume dùng chung chứa tệp trạng thái.
 - Cấu hình vòng đời (`auto_remove='force'`): Container Alpine chỉ sống vừa đủ vài mili-giây để thực thi lệnh `rm -f`, sau đó tự động tiêu hủy hoàn toàn khỏi hệ thống mà không tiêu tốn tài nguyên.
- **Kết quả:** Tệp trạng thái cũ được dọn dẹp triệt để an toàn. Tiến trình cốt lõi của Airflow vẫn được bảo vệ dưới tài khoản người dùng giới hạn (UID 1001/50000) mà không gặp bất kỳ rào cản phân quyền nào.

Điểm rút ra

Thay vì phá vỡ rào cản bảo mật bằng cách cấp đặc quyền cho dịch vụ lõi, hệ thống nên dùng container để triển khai các tác vụ vòng đời ngắn để xử lý các công việc đòi hỏi thẩm



quyền đặc biệt.